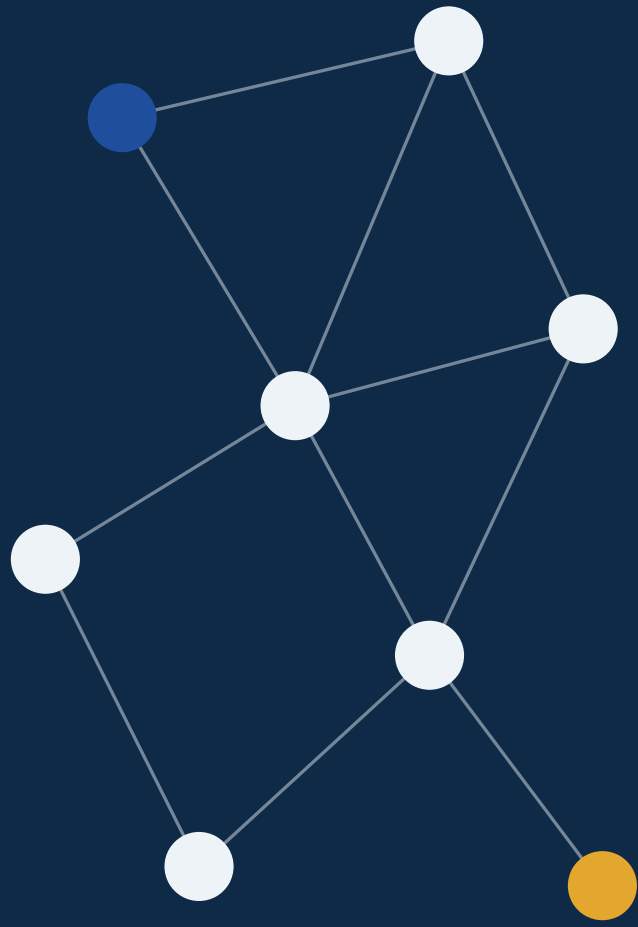
Performance Analysis of AODV in a MANET Using NS-3

with our AH-AODV variant, compared with OLSR and DSDV



Alternate Assessment · Network Programming (23CS7PCNWP)

Tanmay Agarwal (1WA23CS010) · Viamrsh Buyanni (1WA23CS039)

Yeruva Taniya Paul (1WA23CS053) · Vijeth MD (1WA23CS041)

Faculty in charge: Dr. K. Panimozhi, Associate Professor · Department of CSE, BMSCE

Date of submission: 3 November 2026

Why routing in a MANET is hard

A mobile ad hoc network has no access point or router: every node forwards packets for the others, and the routing protocol must keep finding paths as nodes move.



Partitions

A sparse network splits into parts. If no path exists, no protocol can deliver.



Mobility

Moving nodes break links. Routes must be detected as broken and repaired quickly.



Shared channel

Data and routing control packets compete for one radio channel, hop after hop.

Objectives

1. Implement a MANET in ns-3.48 with a UDP client–server application
2. Measure AODV as node density, speed and traffic load change
3. Compare AODV with OLSR and DSDV on identical topologies
4. Propose AH-AODV: fewer Hello messages, same delivery
5. Validate the results and report 95% confidence and significance

560

simulations

14 settings × 4 protocols × 10 random topologies

Three protocols, two approaches

AODV

Reactive · distance vector

Port

UDP port 654

Routes

Finds a route only when needed; buffers packets meanwhile

Control messages

RREQ, RREP, RERR; Hello every 1 s

OLSR

Proactive · link state

Port

UDP port 698

Routes

Routes always in the table; drops packets without a route

Control messages

HELLO every 2 s; TC every 5 s via multipoint relays

DSDV

Proactive · distance vector

Port

UDP port 269

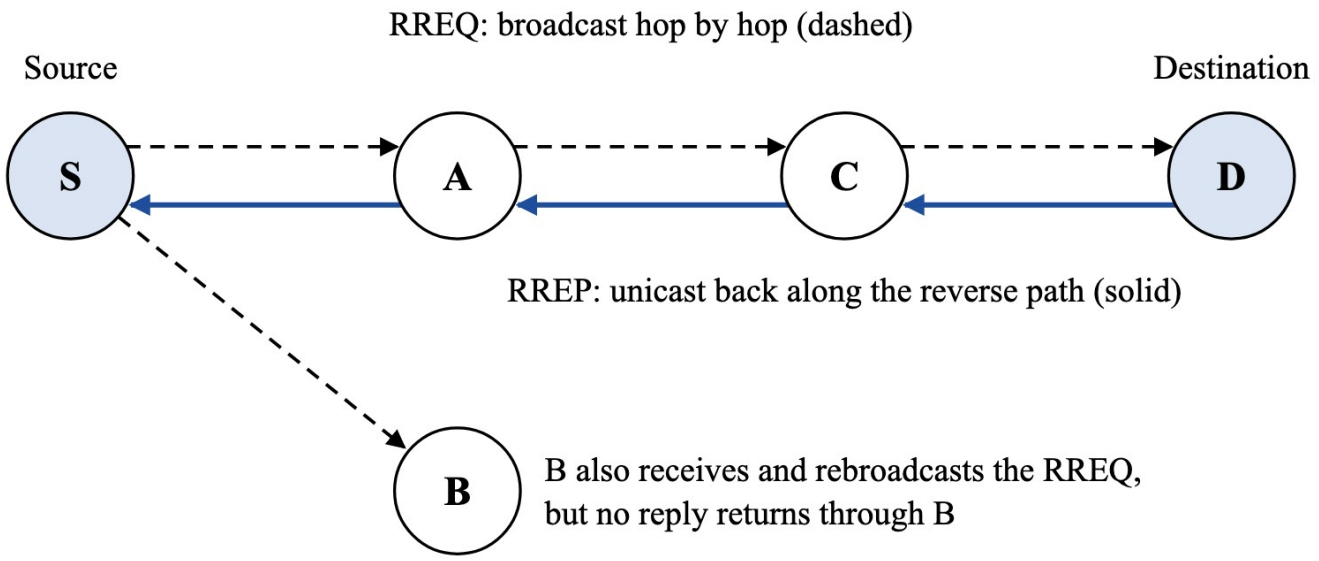
Routes

Routes always in the table; buffers until a route appears

Control messages

Full update every 15 s plus triggered updates

AODV finds routes on demand



RREQ — The source broadcasts a route request; every node records a reverse route and rebroadcasts it.

RREP — The destination replies by unicast along the reverse path, installing the forward route.

RERR — When a link on an active route breaks, a route error tells the nodes using it.

Seq. no. — Destination sequence numbers keep routes fresh and loop-free.

Research gap and problem statement

Research gap

- Results change with density, mobility and load, so rankings depend on the scenario
- Many MANET simulations cannot be repeated or report no spread
- AODV sends a Hello from every node every second, however stable the network; whether an adaptive rule can cut this without losing delivery is unmeasured

Problem statement

Propose AH-AODV, an AODV variant that adapts each node's Hello interval to its neighbourhood and its data traffic, and determine in ns-3.48, with repeated paired runs, whether it cuts AODV's routing overhead without reducing delivery, and how AODV and AH-AODV compare with OLSR and DSDV as density, speed and load change.

Our contribution: AH-AODV (Adaptive-Hello AODV)

In ns-3, every AODV node broadcasts a Hello every second: with 10 nodes that is about 600 of AODV's 630 control packets per run. AH-AODV keeps AODV's route discovery and changes only when Hellos are sent.

Back off when stable

No neighbour joined or left: the next Hello waits 2 s, then 4 s (the maximum)

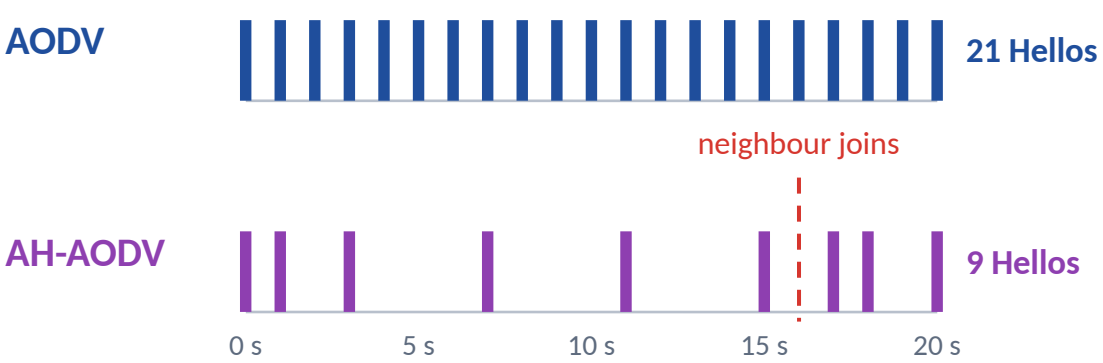
React to change

A neighbour joins or leaves, or a link breaks: back to 1 s at once

Protect active routes

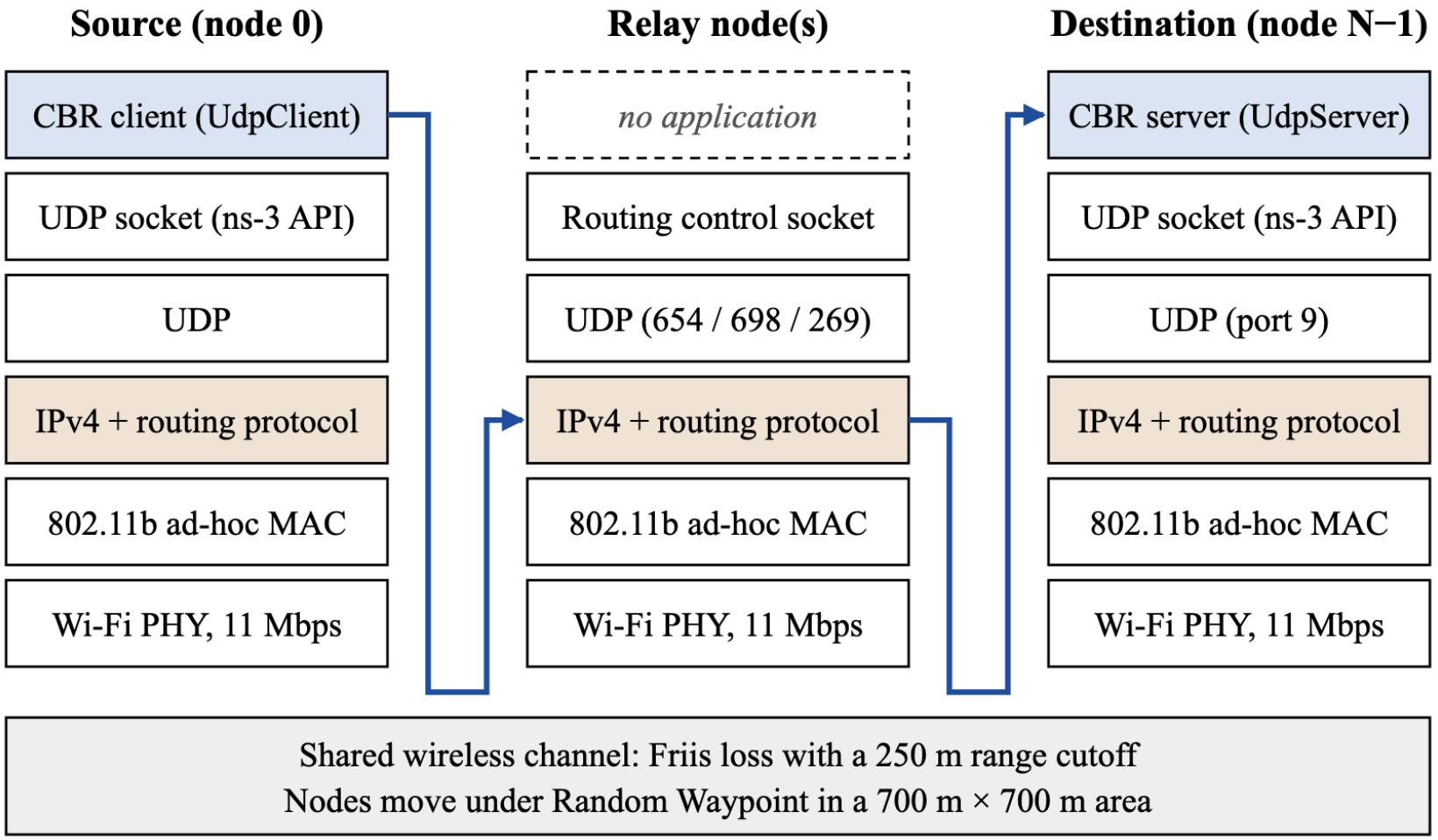
A node that sent, forwarded or received data in the last 3 s keeps 1 s

Hellos sent by one node in 20 s



Each Hello advertises a lifetime of twice its interval, so neighbours keep the node in their tables while it backs off. An idle node in a stable neighbourhood sends about 16 Hellos a minute instead of 60.

System architecture



- Every node runs the same stack: 802.11b ad-hoc Wi-Fi, IPv4 with the routing protocol, UDP and the ns-3 socket API
- Relays forward data at the IPv4 layer using the protocol's routes
- FlowMonitor measures the data flow; packet traces count control packets

Socket programming in the simulation

BSD socket call	ns-3 Socket API	Use in this project
`socket()`	`Socket::CreateSocket(node, UdpSocketFactory)`	Client, server and routing sockets
`bind()`	`Bind(InetSocketAddress(addr, port))`	Server port 9; routing 654 / 698 / 269
`connect()`	`Connect(InetSocketAddress(dst, 9))`	Client fixes the server address
`send()` / `sendto()`	`Send(packet)` / `SendTo(packet, 0, dst)`	Data every 1/λ s; routing messages
`recvfrom()`	`RecvFrom(from)` in a receive callback	Server counts packets; routing reads updates
`setsockopt(SO_BROADCAST)`	`SetAllowBroadcast(true)`	RREQ, HELLO and update broadcasts

ns-3 has no kernel, so these are C++ calls on simulated sockets. Nothing may block in a discrete-event simulator, so reception is event-driven through SetRecvCallback() instead of a blocking recvfrom().

Experimental setup

Simulator	ns-3.48 (C++)
Area, nodes	700 m × 700 m, 10 nodes
Wi-Fi	802.11b ad-hoc, 11 Mbps, 250 m range
Mobility	Random Waypoint, 5 m/s, 0 s pause
Traffic	UDP CBR, 512 B, 4 packets/s, node 0 → N−1
Time	60 s (traffic 2–59 s)
Protocols	AODV, AH-AODV (ours), OLSR, DSDV
Repetitions	10 runs; run k = same topology for all protocols

A **Number of nodes**

5, 10, 15, 20

B **Node speed**

1, 5, 10, 15 m/s

C **Traffic load**

10 – 800 packets/s

One parameter changes at a time; the others keep their defaults.

What we measured

Packet delivery ratio

Received ÷ sent data packets

Throughput

Delivered bits per second at the IP layer

End-to-end delay

Mean transit time of delivered packets

Routing overhead

Routing control packets sent by all nodes in a run

Source drops

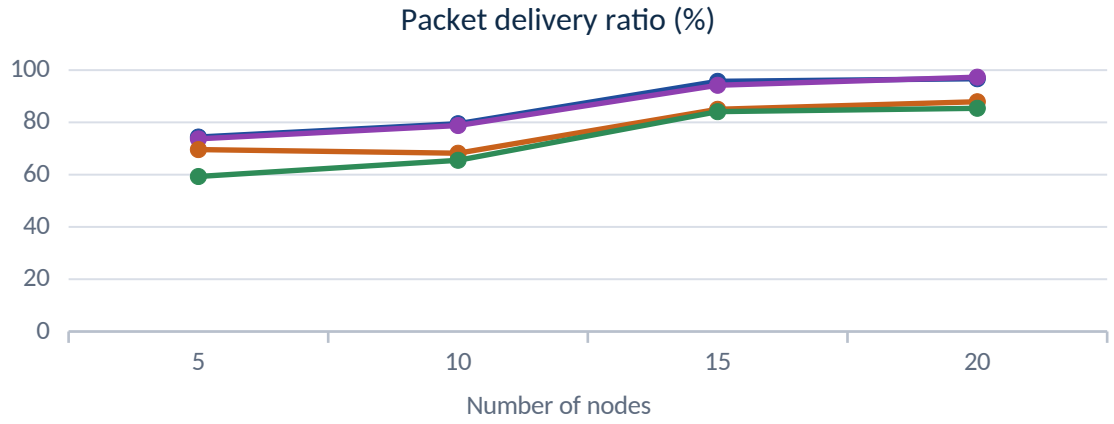
Packets dropped at the source for lack of a route

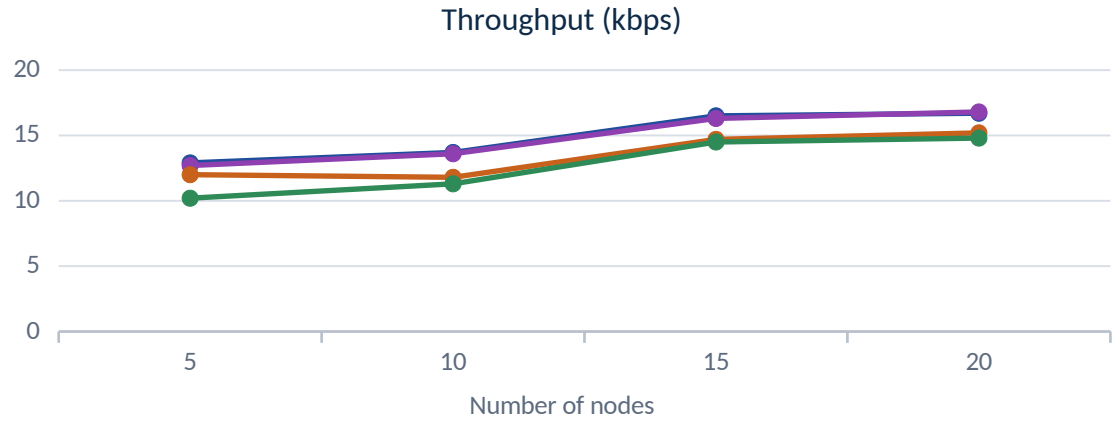
Overhead is counted with the Ipv4L3Protocol Tx trace because FlowMonitor ignores broadcast packets; sent data packets are counted at the client because OLSR drops packets before FlowMonitor sees them.

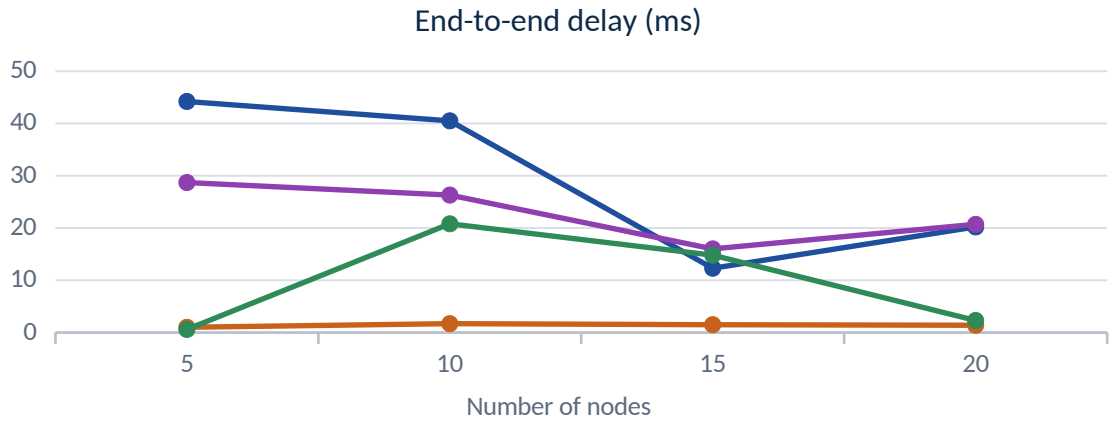
Experiment A: more nodes help every protocol

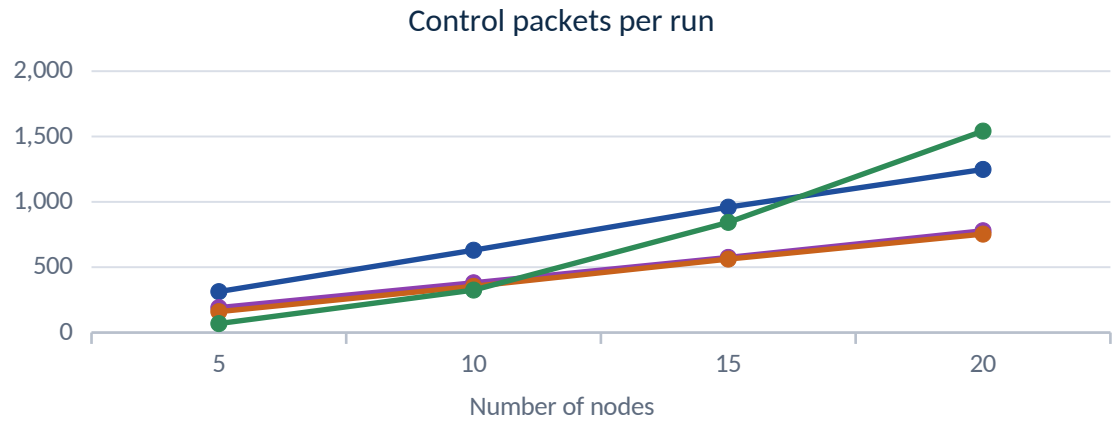
AODV AH-AODV OLSR DSDV

Mean of 10 runs per point · 5 m/s, 4 packets/s





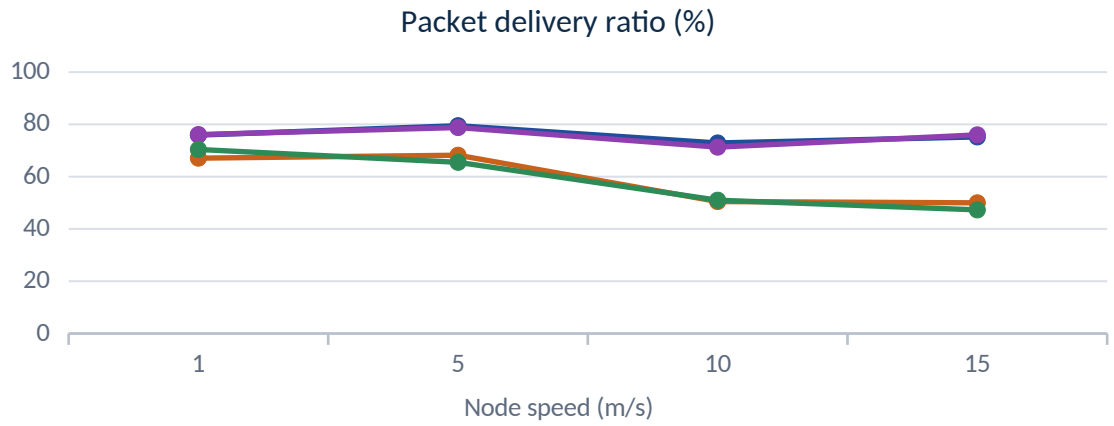


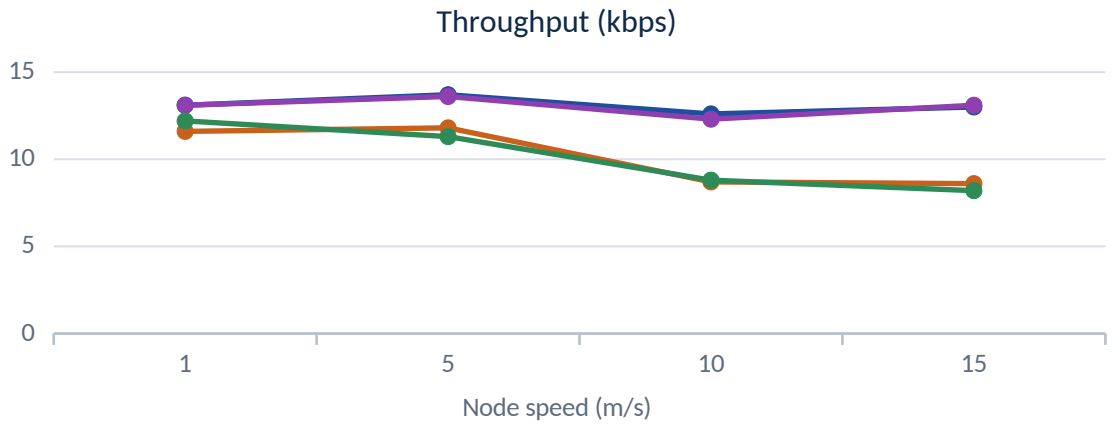


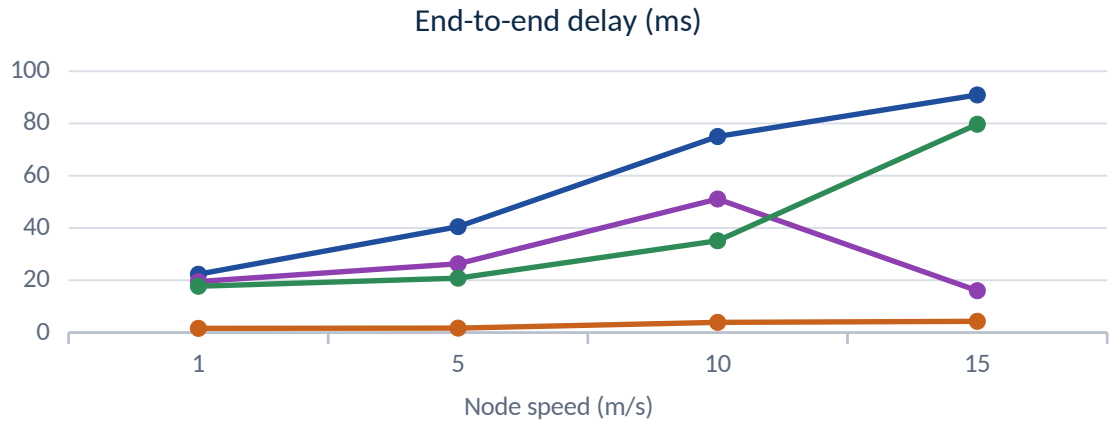
Experiment B: under mobility, AODV holds its delivery

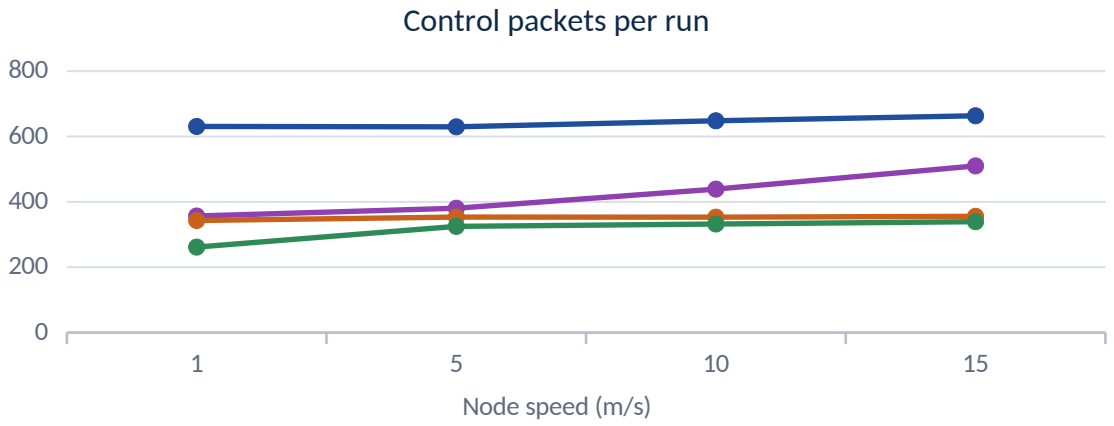
AODV AH-AODV OLSR DSDV

Mean of 10 runs per point · 10 nodes, 4 packets/s





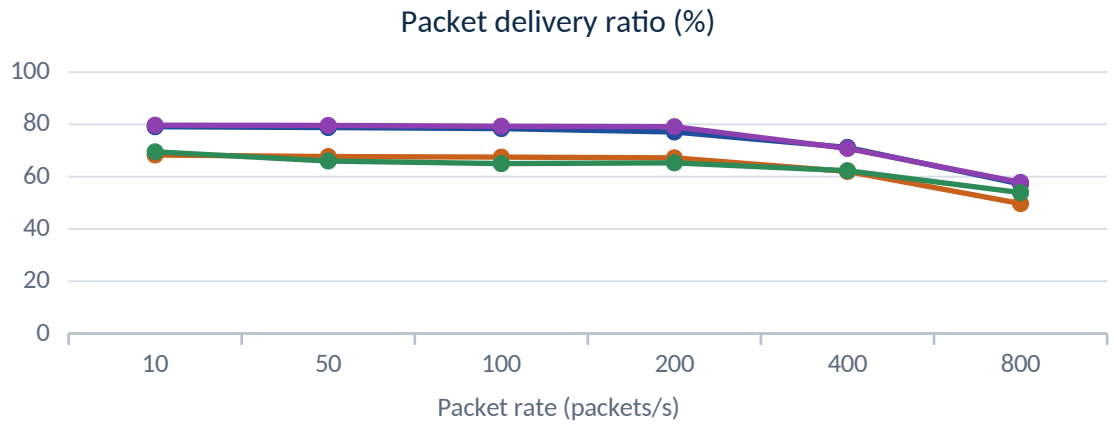


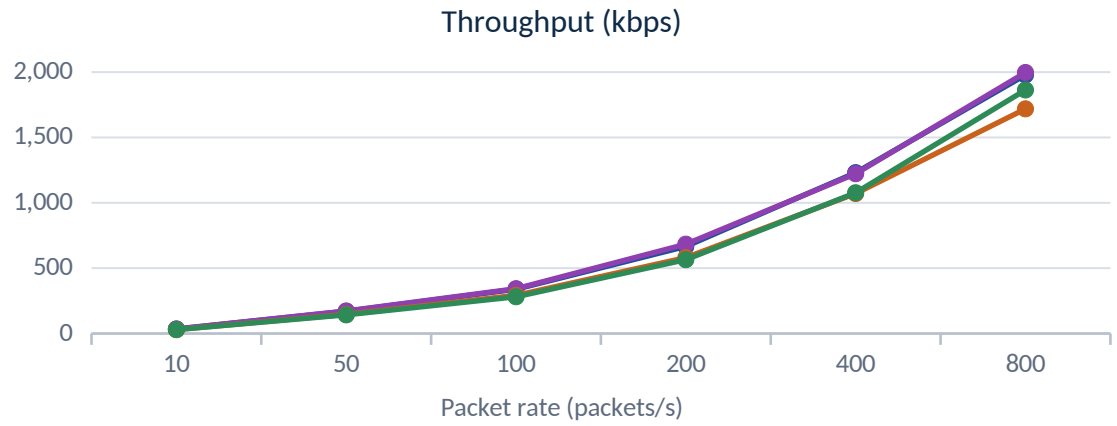


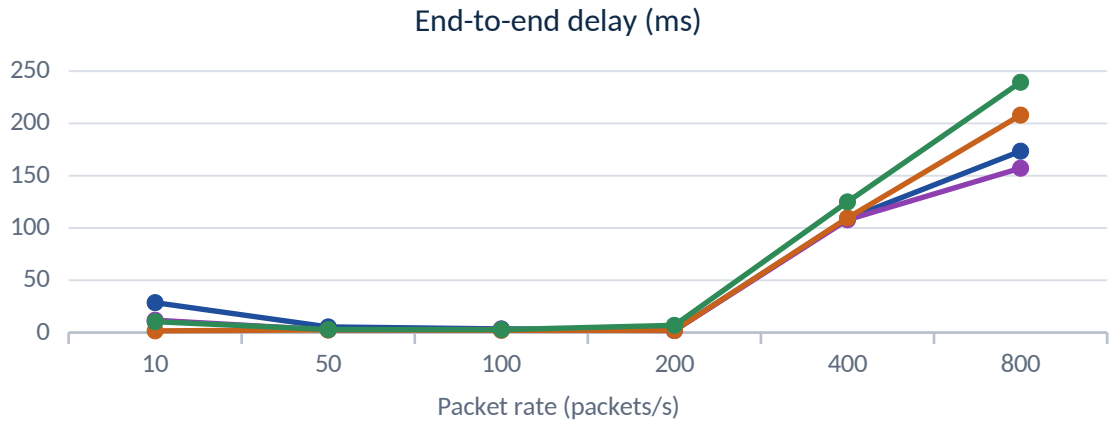
Experiment C: heavy load congests every protocol

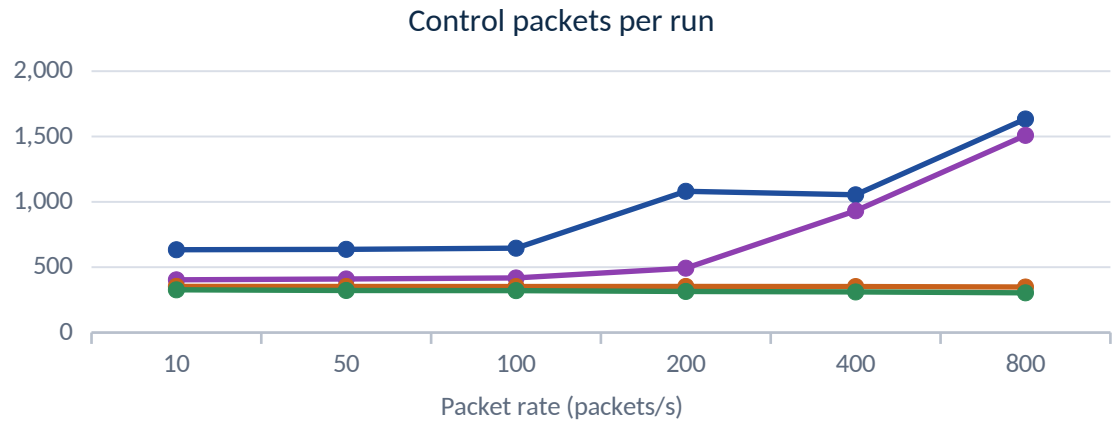
AODV AH-AODV OLSR DSDV

Mean of 10 runs per point · 10 nodes, 5 m/s

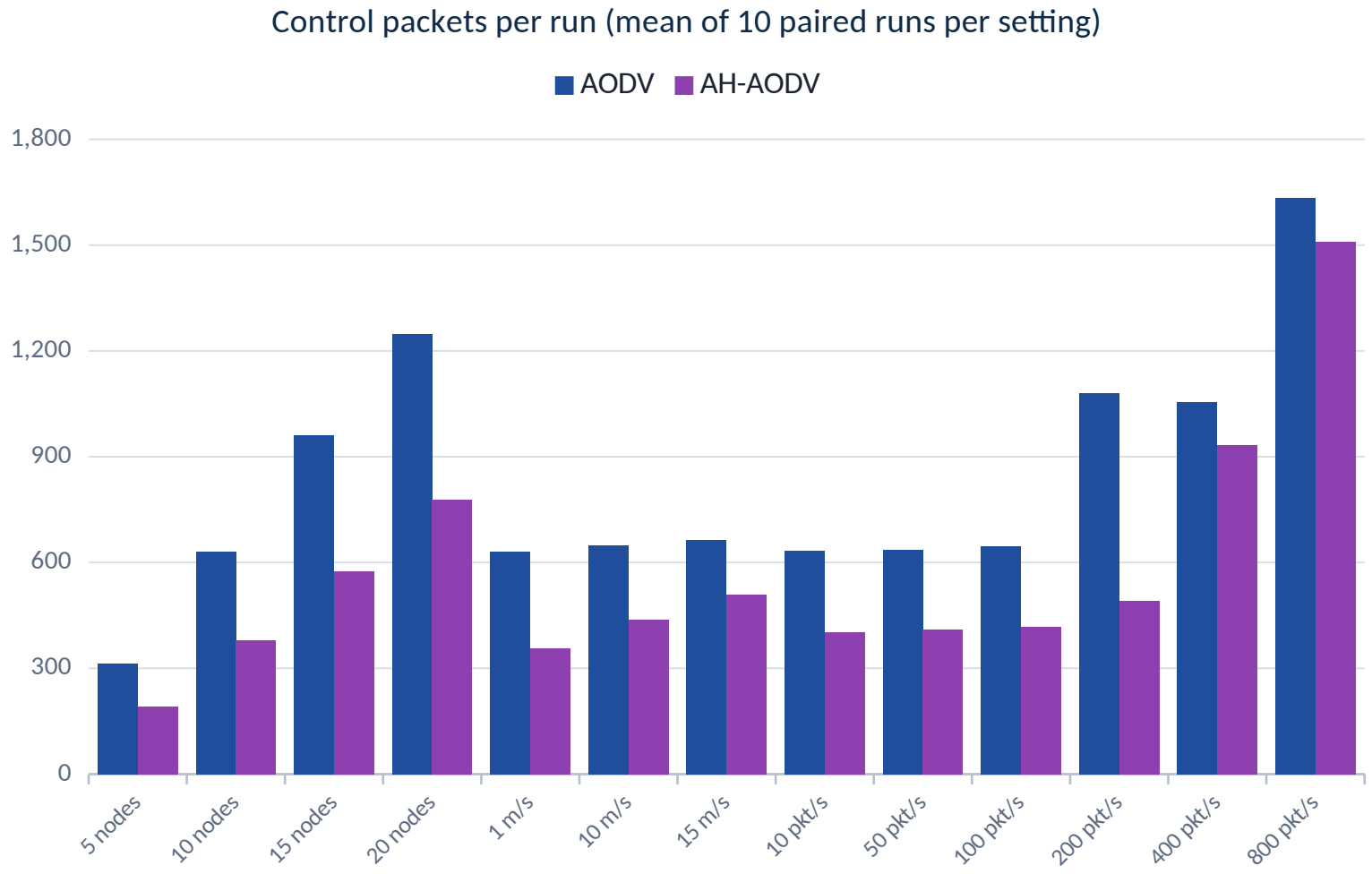








AH-AODV: 31% fewer control packets, same delivery



−31%

control packets: 829 → 569 per run on average; significantly fewer in 10 of 13 settings

78.0%

delivery ratio, against 77.9% for AODV: no significant difference in any setting

36 ms

mean delay, against 48 ms for AODV (significant only at 200 packets/s)

Comparative summary

77.9%

AODV: mean PDR over 13 settings

78.0%

AH-AODV: mean PDR over 13 settings

66.2%

OLSR: mean PDR over 13 settings

65.0%

DSDV: mean PDR over 13 settings

AODV and AH-AODV deliver significantly more than OLSR (12 and 11 of 13 settings) and DSDV (9 of 13), never less

AODV pays with about twice the overhead; AH-AODV cuts it by 31% with the same delivery

Paired by topology, so differences are significant despite large run-to-run spread

Conclusion and future work

Findings

- AODV delivered the most: 77.9% vs 66.2% (OLSR) and 65.0% (DSDV); 75% vs about 50% at 15 m/s
- Its cost is about twice the overhead, mostly fixed 1 s Hellos
- Our AH-AODV cut control packets by 31% with the same delivery (78.0%)
- Single runs ranged from 0% to 100%: repeated, paired runs are essential

Future work

- AH-AODV with many flows and a fading radio model
- Congestion-aware route discovery for heavy load
- Tune protocol timers; more runs and TCP traffic
- Energy use and black hole attack resilience

Thank you — questions?